LABSHEET-9

Collections

Q1. Student List

Create an ArrayList<String> containing ten student names. Display all names and the total size.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();
        students.add("Aryan  ");
        students.add("Rahul");
        students.add("Priya");
        students.add("Amit");
        students.add("Neha");
        students.add("Rohan");
        students.add("Sneha");
        students.add("Karan");
        students.add("Anjali");
        students.add("Vikas");

        System.out.println("Student Names:");
        for (String name : students) {
            System.out.println(name);
        }
        System.out.println("Total Size: " + students.size());
    }
}
```

OUTPUT:-

```
Student Names:
Aryan
Rahul
Priya
Amit
Neha
Rohan
Sneha
Karan
Anjali
Vikas
Total Size: 10
```

Q2. Marks List

Create an ArrayList<Integer> containing five marks. Display the first, last and middle elements using get().

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> marks = new ArrayList<>();
        marks.add(78);
        marks.add(85);
        marks.add(92);
        marks.add(67);
        marks.add(88);

        System.out.println("First Element: " + marks.get(0));
        System.out.println("Middle Element: " + marks.get(marks.size() / 2));
        System.out.println("Last Element: " + marks.get(marks.size() - 1));
    }
}
```

OUTPUT:-

```
First Element: 78
Middle Element: 92
Last Element: 88
```

Q3. Update Student Name

Create an ArrayList<String>. Replace the second student name using set() and remove the fourth student.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();
        students.add("Aryan  ");
        students.add("Rahul");
        students.add("Priya");
        students.add("Amit");
        students.add("Neha");

        System.out.println("Original List: " + students);

        students.set(1, "Rohan");          // replace 2nd student
        System.out.println("After set(1, \"Rohan\"): " + students);

        students.remove(3);                // remove 4th student
        System.out.println("After remove(3): " + students);
    }
}
```

OUTPUT:-

```
Original List: [Aryan  , Rahul, Priya, Amit, Neha]
After set(1, "Rohan"): [Aryan  , Rohan, Priya, Amit, Neha]
After remove(3): [Aryan  , Rohan, Priya, Neha]
```

Q4. Unique Departments

Create a HashSet<String> containing department names with duplicates. Display the resulting unique departments.

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> departments = new HashSet<>();
        departments.add("CSE");
        departments.add("ECE");
        departments.add("CSE");
        departments.add("ME");
        departments.add("ECE");
        departments.add("Civil");
        departments.add("ME");

        System.out.println("Unique Departments:");
        for (String d : departments) {
            System.out.println(d);
        }
        System.out.println("Total Unique Departments: " + departments.size());
    }
}
```

OUTPUT:-

```
Unique Departments:
Civil
CSE
ECE
ME
Total Unique Departments: 4
```

Q5. Sorted Roll Numbers

Create a TreeSet<Integer> containing ten roll numbers in random order. Display them in sorted order.

```
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> rollNumbers = new TreeSet<>();
        rollNumbers.add(109);
        rollNumbers.add(103);
```

```
        rollNumbers.add(107);
        rollNumbers.add(101);
        rollNumbers.add(110);
        rollNumbers.add(105);
        rollNumbers.add(102);
        rollNumbers.add(108);
        rollNumbers.add(104);
        rollNumbers.add(106);

        System.out.println("Roll Numbers in Sorted Order:");
        for (int r : rollNumbers) {
            System.out.println(r);
        }
    }
}
```

OUTPUT:-

```
Roll Numbers in Sorted Order:
101
102
103
104
105
106
107
108
109
110
```

Q6. ArrayList Searching

Create an ArrayList<String> of course names. Ask the user for a course and use contains() to check whether it exists.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> courses = new ArrayList<>();
        courses.add("BCA");
        courses.add("BBA");
        courses.add("BTech");
        courses.add("MCA");
        courses.add("MBA");

        Scanner sc = new Scanner(System.in);
        System.out.println("Available Courses: " + courses);
        System.out.print("Enter a course to search: ");
        String course = sc.nextLine();

        if (courses.contains(course)) {
            System.out.println(course + " exists in the list.");
        } else {
            System.out.println(course + " does not exist in the list.");
        }
        sc.close();
    }
}
```

OUTPUT:-

```
Available Courses: [BCA, BBA, BTech, MCA, MBA]
Enter a course to search: MCA
MCA exists in the list.
```

Q7. Collections Utility Methods

Create an ArrayList<Integer> and demonstrate Collections.sort(), reverse(), max(), min() and frequency().

```
import java.util.ArrayList;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
```

```
        ArrayList<Integer> list = new ArrayList<>();
        list.add(40);
        list.add(10);
        list.add(30);
        list.add(20);
        list.add(10);
        list.add(50);

        System.out.println("Original List: " + list);

        Collections.sort(list);
        System.out.println("After sort(): " + list);

        Collections.reverse(list);
        System.out.println("After reverse(): " + list);

        System.out.println("Maximum: " + Collections.max(list));
        System.out.println("Minimum: " + Collections.min(list));
        System.out.println("Frequency of 10: " + Collections.frequency(list, 10));
    }
}
```

OUTPUT:-

```
Original List: [40, 10, 30, 20, 10, 50]
After sort(): [10, 10, 20, 30, 40, 50]
After reverse(): [50, 40, 30, 20, 10, 10]
Maximum: 50
Minimum: 10
Frequency of 10: 2
```

Q8. LinkedList Student Queue

Create a LinkedList<String> representing students waiting for a lab. Use addFirst(), addLast(), removeFirst() and removeLast().

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> queue = new LinkedList<>();
        queue.add("Rahul");
        queue.add("Priya");
        System.out.println("Initial Queue: " + queue);

        queue.addFirst("Aryan  ");
        System.out.println("After addFirst(\"Aryan  \"): " + queue);

        queue.addLast("Amit");
        System.out.println("After addLast(\"Amit\"): " + queue);

        System.out.println("Removed First: " + queue.removeFirst());
        System.out.println("Queue: " + queue);

        System.out.println("Removed Last: " + queue.removeLast());
        System.out.println("Final Queue: " + queue);
    }
}
```

OUTPUT:-

```
Initial Queue: [Rahul, Priya]
After addFirst("Aryan  "): [Aryan  , Rahul, Priya]
After addLast("Amit"): [Aryan  , Rahul, Priya, Amit]
Removed First: Aryan
Queue: [Rahul, Priya, Amit]
Removed Last: Amit
Final Queue: [Rahul, Priya]
```

Q9. ListIterator Modification

Create a List<String> and use ListIterator to traverse it. Use set() to replace one element and add() to insert an element during iteration.

```
import java.util.ArrayList;
import java.util.List;
```

```
import java.util.ListIterator;

public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");
        list.add("HTML");

        System.out.println("Original List: " + list);

        ListIterator<String> it = list.listIterator();
        while (it.hasNext()) {
            String item = it.next();
            if (item.equals("Python")) {
                it.set("JavaScript");      // replace element
            }
            if (item.equals("C++")) {
                it.add("SQL");             // insert after C++
            }
        }

        System.out.println("Modified List: " + list);
    }
}
```

OUTPUT:-

```
Original List: [Java, Python, C++, HTML]
Modified List: [Java, JavaScript, C++, SQL, HTML]
```

Q10. List vs Set Experiment

Create an ArrayList<Integer> and HashSet<Integer> using the same input values. Display both and explain the difference in duplicates and ordering.

```
import java.util.ArrayList;
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        int[] values = {40, 10, 30, 10, 20, 40, 50};

        ArrayList<Integer> list = new ArrayList<>();
        HashSet<Integer> set = new HashSet<>();

        for (int v : values) {
            list.add(v);
            set.add(v);
        }

        System.out.println("Input Values: 40, 10, 30, 10, 20, 40, 50");
        System.out.println("ArrayList: " + list);
        System.out.println("HashSet  : " + set);

        System.out.println("\nDifference:");
        System.out.println("1. ArrayList allows duplicates (size = "
                + list.size() + "), HashSet does not (size = "
                + set.size() + ").");
        System.out.println("2. ArrayList keeps insertion order, "
                + "HashSet does not guarantee any order.");
    }
}
```

OUTPUT:-

```
Input Values: 40, 10, 30, 10, 20, 40, 50
ArrayList: [40, 10, 30, 10, 20, 40, 50]
HashSet  : [50, 20, 40, 10, 30]

Difference:
1. ArrayList allows duplicates (size = 7), HashSet does not (size = 5).
2. ArrayList keeps insertion order, HashSet does not guarantee any order.
```

Explanation: ArrayList stores all 7 values in insertion order, including the duplicates (40 and 10 appear twice). HashSet stores only the 5 unique values and does not guarantee any particular order (order depends on hash codes).

Q11. Sorted Student IDs

Create a TreeSet<Integer> for student IDs. Add duplicate IDs and display the unique sorted IDs. Also demonstrate first(), last(), higher() and lower().

```
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> ids = new TreeSet<>();
        ids.add(105);
        ids.add(102);
        ids.add(108);
        ids.add(102);   // duplicate
        ids.add(101);
        ids.add(105);   // duplicate
        ids.add(110);

        System.out.println("Unique Sorted IDs: " + ids);
        System.out.println("first(): " + ids.first());
        System.out.println("last(): " + ids.last());
        System.out.println("higher(105): " + ids.higher(105));
        System.out.println("lower(105): " + ids.lower(105));
    }
}
```

OUTPUT:-

```
Unique Sorted IDs: [101, 102, 105, 108, 110]
first(): 101
last(): 110
higher(105): 108
lower(105): 102
```

Q12. Hospital Patient List

Create an ArrayList<String> of patient names and a HashSet<String> of unique patient IDs. Add, remove, search and display the records.

```
import java.util.ArrayList;
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> patientNames = new ArrayList<>();
        HashSet<String> patientIds = new HashSet<>();

        // Add
        patientNames.add("Rahul");
        patientNames.add("Priya");
        patientNames.add("Amit");
        patientIds.add("P101");
        patientIds.add("P102");
        patientIds.add("P103");
        patientIds.add("P102");   // duplicate ignored
        System.out.println("After Adding:");
        System.out.println("Patient Names: " + patientNames);
        System.out.println("Patient IDs  : " + patientIds);

        // Remove
        patientNames.remove("Priya");
        patientIds.remove("P102");
        System.out.println("\nAfter Removing Priya / P102:");
        System.out.println("Patient Names: " + patientNames);
        System.out.println("Patient IDs  : " + patientIds);

        // Search
        System.out.println("\nSearch Results:");
        System.out.println("Is Amit a patient? " + patientNames.contains("Amit"));
        System.out.println("Is P102 present? " + patientIds.contains("P102"));
```

```
        System.out.println("Is P103 present? " + patientIds.contains("P103"));

        // Display
        System.out.println("\nFinal Records:");
        System.out.println("Total Patients: " + patientNames.size());
        System.out.println("Total Unique IDs: " + patientIds.size());
    }
}
```

OUTPUT:-

```
After Adding:
Patient Names: [Rahul, Priya, Amit]
Patient IDs  : [P103, P102, P101]

After Removing Priya / P102:
Patient Names: [Rahul, Amit]
Patient IDs  : [P103, P101]

Search Results:
Is Amit a patient? true
Is P102 present? false
Is P103 present? true

Final Records:
Total Patients: 2
Total Unique IDs: 2
```

Q13. ListIterator Report

Create an ArrayList<String> of five course names. Use ListIterator to display the list forward and backward, and demonstrate nextIndex() and previousIndex().

```
import java.util.ArrayList;
import java.util.ListIterator;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> courses = new ArrayList<>();
        courses.add("BCA");
        courses.add("BBA");
        courses.add("BTech");
        courses.add("MCA");
        courses.add("MBA");

        ListIterator<String> it = courses.listIterator();

        System.out.println("Forward Traversal:");
        while (it.hasNext()) {
            System.out.println("nextIndex = " + it.nextIndex() + " -> " + it.next());
        }

        System.out.println("\nBackward Traversal:");
        while (it.hasPrevious()) {
            System.out.println("previousIndex = " + it.previousIndex()
                    + " -> " + it.previous());
        }
    }
}
```

OUTPUT:-

```
Forward Traversal:
nextIndex = 0 -> BCA
nextIndex = 1 -> BBA
nextIndex = 2 -> BTech
nextIndex = 3 -> MCA
nextIndex = 4 -> MBA

Backward Traversal:
previousIndex = 4 -> MBA
previousIndex = 3 -> MCA
previousIndex = 2 -> BTech
previousIndex = 1 -> BBA
previousIndex = 0 -> BCA
```

Q14. ArrayList vs LinkedList Lab

Create equivalent ArrayList and LinkedList programs. Perform insertion, deletion, get(), size() and iteration operations. Record your observations in a comparison table.

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

public class Main {
    static void perform(List<String> list, String type) {
        System.out.println("===== " + type + " =====");
        list.add("Rahul");
        list.add("Priya");
        list.add("Amit");
        System.out.println("After insertion : " + list);

        list.add(1, "Aryan  ");
        System.out.println("Insert at index 1: " + list);

        list.remove("Amit");
        System.out.println("After deletion  : " + list);

        System.out.println("get(2)          : " + list.get(2));
        System.out.println("size()          : " + list.size());

        System.out.print("Iteration       : ");
        for (String s : list) {
            System.out.print(s + " ");
        }
        System.out.println("\n");
    }

    public static void main(String[] args) {
        perform(new ArrayList<>(), "ArrayList");
        perform(new LinkedList<>(), "LinkedList");
    }
}
```

OUTPUT:-

```
===== ArrayList =====
After insertion : [Rahul, Priya, Amit]
Insert at index 1: [Rahul, Aryan  , Priya, Amit]
After deletion  : [Rahul, Aryan  , Priya]
get(2)          : Priya
size()          : 3
Iteration       : Rahul Aryan   Priya

===== LinkedList =====
After insertion : [Rahul, Priya, Amit]
Insert at index 1: [Rahul, Aryan  , Priya, Amit]
After deletion  : [Rahul, Aryan  , Priya]
get(2)          : Priya
size()          : 3
Iteration       : Rahul Aryan   Priya
```

Comparison Table:-

Operation	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Insertion at end	Fast	Fast
Insertion in middle / start	Slow (elements shift)	Fast (only links change)
Deletion	Slow (elements shift)	Fast once node is found
get(index)	Fast (direct access)	Slow (must traverse nodes)
size()	Fast	Fast
Iteration	Fast	Fast
Memory usage	Less	More (extra node pointers)
Best used for	Frequent reading / searching	Frequent insertion / deletion

Q15. Mini Student Collection Management System

Create a student collection management program. Use ArrayList for maintaining student records in insertion order, HashSet for unique student IDs, and TreeSet for sorted student IDs. Provide options to add, remove, search, display, count, and iterate students. Use ListIterator for forward/backward traversal where appropriate. Demonstrate the difference between List, Set and SortedSet.

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.ListIterator;
import java.util.Scanner;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();   // List  - insertion order
        HashSet<Integer> idSet = new HashSet<>();         // Set   - unique IDs
        TreeSet<Integer> sortedIds = new TreeSet<>();     // SortedSet - sorted IDs
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n===== Student Collection Management =====");
            System.out.println("1. Add Student");
            System.out.println("2. Remove Student");
            System.out.println("3. Search Student");
            System.out.println("4. Display Students");
            System.out.println("5. Count Students");
            System.out.println("6. Iterate (Forward/Backward)");
            System.out.println("7. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter ID: ");
                    int id = sc.nextInt();
                    System.out.print("Enter Name: ");
                    String name = sc.next();
                    if (idSet.add(id)) {
                        sortedIds.add(id);
                        students.add(id + " - " + name);
                        System.out.println("Student added.");
                    } else {
                        System.out.println("ID already exists. Duplicate not allowed.");
                    }
                    break;

                case 2:
                    System.out.print("Enter ID to remove: ");
                    int rid = sc.nextInt();
                    if (idSet.remove(rid)) {
                        sortedIds.remove(rid);
                        students.removeIf(s -> s.startsWith(rid + " - "));
                        System.out.println("Student removed.");
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;

                case 3:
                    System.out.print("Enter ID to search: ");
                    int sid = sc.nextInt();
                    if (idSet.contains(sid)) {
                        for (String s : students) {
                            if (s.startsWith(sid + " - ")) {
                                System.out.println("Found: " + s);
                            }
                        }
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;
```

```
                case 4:
                    System.out.println("List (insertion order) : " + students);
                    System.out.println("HashSet (unique IDs)   : " + idSet);
                    System.out.println("TreeSet (sorted IDs)   : " + sortedIds);
                    break;

                case 5:
                    System.out.println("Total Students: " + students.size());
                    System.out.println("Unique IDs    : " + idSet.size());
                    break;

                case 6:
                    ListIterator<String> it = students.listIterator();
                    System.out.println("Forward:");
                    while (it.hasNext()) {
                        System.out.println(it.next());
                    }
                    System.out.println("Backward:");
                    while (it.hasPrevious()) {
                        System.out.println(it.previous());
                    }
                    break;

                case 7:
                    System.out.println("Exiting program.");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 7);

        sc.close();
    }
}
```

OUTPUT:-

```
===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 1
Enter ID: 103
Enter Name: Rahul
Student added.

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 1
Enter ID: 101
Enter Name: Priya
Student added.

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 1
Enter ID: 105
```

```
Enter Name: Amit
Student added.

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 1
Enter ID: 101
Enter Name: Neha
ID already exists. Duplicate not allowed.

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 4
List (insertion order) : [103 - Rahul, 101 - Priya, 105 - Amit]
HashSet (unique IDs)   : [101, 103, 105]
TreeSet (sorted IDs)   : [101, 103, 105]

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 3
Enter ID to search: 105
Found: 105 - Amit

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 5
Total Students: 3
Unique IDs    : 3

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 6
Forward:
103 - Rahul
101 - Priya
105 - Amit
Backward:
105 - Amit
101 - Priya
103 - Rahul

===== Student Collection Management =====
1. Add Student
```

```
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 2
Enter ID to remove: 103
Student removed.

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 4
List (insertion order) : [101 - Priya, 105 - Amit]
HashSet (unique IDs)   : [101, 105]
TreeSet (sorted IDs)   : [101, 105]

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 5
Total Students: 2
Unique IDs    : 2

===== Student Collection Management =====
1. Add Student
2. Remove Student
3. Search Student
4. Display Students
5. Count Students
6. Iterate (Forward/Backward)
7. Exit
Enter choice: 7
Exiting program.
```